

HTTP module - creating basic servers

Absolutely! Here's a detailed explanation of the **Node.js** `http` module and how to create **basic servers** using it.

Node.js `http` Module – Overview

The `http` module allows you to **create web servers** and **handle HTTP requests and responses** directly, without needing any external libraries.

It's the core of most Node.js web frameworks like **Express.js**, but using it directly helps you understand how the server works at a low level.

◆ 1. Importing `http`

```
// CommonJS
const http = require('http');

// ES Module
import http from 'http';
```

2. Creating a Basic HTTP Server

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Set response header
  res.writeHead(200, { 'Content-Type': 'text/plain' });

  // Write response body
  res.end('Hello, World!');
```

```
});  
  
server.listen(3000, () => {  
  console.log('Server is running at http://localhost:3000');  
});
```

Explanation:

- `http.createServer()` takes a **callback function** with parameters `req` (request) and `res` (response).
- `res.writeHead()` sets the **status code** and **headers**.
- `res.end()` sends the response to the client and ends the request.

3. Accessing Request Data

```
const server = http.createServer((req, res) => {  
  console.log('Method:', req.method);  
  console.log('URL:', req.url);  
  
  res.writeHead(200, { 'Content-Type': 'text/plain' });  
  res.end('Request received!');  
});
```

You can inspect the request method, URL, and headers with:

- `req.method`
- `req.url`
- `req.headers`

4. Simple Routing Example

```
const server = http.createServer((req, res) => {  
  if (req.url === '/') {
```

```
res.writeHead(200, { 'Content-Type': 'text/html' });
res.end('<h1>Home Page</h1>');
} else if (req.url === '/about') {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end('<h1>About Page</h1>');
} else {
  res.writeHead(404, { 'Content-Type': 'text/html' });
  res.end('<h1>404 Not Found</h1>');
}
});
```



5. Serving JSON Data

```
const data = { message: 'Hello, JSON!' };

const server = http.createServer((req, res) => {
  if (req.url === '/api') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(data));
  }
});
```



6. Listening on a Port

```
server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

You can use other ports like 4000, 8080, etc. Ensure they are not blocked or already used.



Key Concepts

Concept	Description
<code>req.method</code>	HTTP method (GET, POST, etc.)
<code>req.url</code>	Requested URL path
<code>res.writeHead()</code>	Sets status code and headers
<code>res.end()</code>	Sends the response and ends the request
<code>createServer()</code>	Creates an HTTP server

Full Example

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.setHeader('Content-Type', 'text/html');

  if (req.url === '/') {
    res.statusCode = 200;
    res.end('<h1>Welcome to the Home Page</h1>');
  } else {
    res.statusCode = 404;
    res.end('<h1>Page Not Found</h1>');
  }
});

server.listen(5000, () => {
  console.log('Server running at http://localhost:5000');
});
```

Let me know if you want to learn how to handle form submissions, POST data, or serve static files using `http` !